PTL: Principal-Teacher-Loop

A New Proposed Architecture for Verified Adaptive Reasoning
in Recurrent Depth Transformers

Elliot Williams

April 21, 2026

NEW PROPOSED MODEL | Not yet published in academic literature | Submitted for Anthropic Research Consideration

---

Abstract

This paper introduces PTL, the Principal-Teacher-Loop, a new proposed model for how recurrent transformer architectures can reason adaptively and safely across many inference loops without hidden state degradation. It has not appeared in the academic literature in its unified form. PTL combines four well-studied but separately developed mechanisms into a single inference-time system: adaptive halting, per-loop process reward grading, structured scratchpad delegation, and a hierarchical principal meta-verifier that monitors grader calibration at runtime. The central claim is that none of these mechanisms alone is sufficient. Stability without verification still runs blind. Grading without delegation still lets degraded outputs propagate. Halting without certified handoff still loses what was resolved. PTL proposes a way of thinking about looped reasoning that treats each loop as an accountable agent in a larger verification chain, not just a repeated computation step.

---

1. The Problem This Architecture Solves

In April 2026, researchers from UC San Diego and Together AI published Parcae, one of the first stable architectures for looped language models. The central result was significant: a 770 million parameter Parcae model matched the quality of a standard transformer with 1.3 billion parameters. The insight driving this was that reasoning quality can scale through inference-time loops rather than through stored parameters. The same transformer block runs again and again, each pass refining the hidden state, until the model produces its output. This is called depth extrapolation: training on four loops and running on sixteen.

Parcae solved the training stability problem that had broken all previous looped architectures. By recasting the recurrent update as a dynamical system and constraining its spectral norm so the system

is always stable by construction, the researchers finally made looped transformers reliable enough to derive predictable scaling laws from. This was a genuine breakthrough.

But Parcae itself acknowledged the boundary of what it solved. In its own limitations section, the paper stated plainly that it left dynamic halting and token-level adaptive recurrence unexplored, and that it remained unknown whether integrating halting policies with Parcae preserves stability and improves compute-quality trade-offs. PTL is a direct response to that open question.

The problem PTL addresses is what happens inside a long inference-time loop sequence even when the architecture is stable. Two independent research groups have now confirmed, empirically, that it is a real and serious problem. Kohli et al. (April 2026) named it overthin king in the context of compositional generalization, showing that across all models and tasks, performance increases with inference iterations until reaching a peak and then consistently declines as iterations continue. More loops do not always mean better reasoning. After a certain point, they make it worse.

Fu et al. (November 2025) identified the same phenomenon at the token level, calling it latent overthinking. They showed that finetuning a model to always apply two iterations per token yielded more errors than the single-iteration baseline, because easy tokens that were already correct were being revised into wrong answers by the extra computation. Their proposed solution, Think-at-Hard, uses a lightweight neural decider to trigger deeper iterations only on tokens that are likely wrong after the first pass. This is a valuable step, but it operates at the token level. PTL proposes that the same principle needs to operate at the reasoning level: not just which tokens need more computation, but which subproblems have been resolved, which need more deliberation, and whether the assessment of completion is itself trustworthy.

Stability is a prerequisite. It is not a solution. A stable hidden state that is silently overthinking, gradually drifting past the correct answer and into noise, is still a failure. PTL is the architecture that addresses what comes after stability.

2. The Overthinking Problem in Depth

To understand why PTL proposes what it does, it helps to understand precisely how overthinking manifests in looped transformers and why existing partial solutions leave a gap.

When a recurrent transformer runs more loops than needed to solve a problem, the hidden state does not simply freeze at the correct answer and wait. It keeps updating. The transformer blocks are still running, still pushing the representation toward new configurations based on patterns learned during training. If the correct answer was reached at loop six and the model continues to loop twelve, those

six additional loops are not free. They are actively corrupting what loop six got right.

This is not just a compute efficiency problem. It is a reliability problem. A model that overthinks does not know it is overthinking. It has no internal signal that says the representation has passed its peak. Without that signal, it has no way to stop, no way to preserve what was correct, and no way to prevent the corruption from propagating to the final output.

Think-at-Hard addresses this at the token level by learning a decider that flags individual tokens as hard or easy. This is a useful granularity for next-token prediction tasks. But recurrent depth transformers are increasingly being used for multi-hop compositional reasoning, where the unit of interest is not a token but a subproblem. The question is not just whether a given token is likely to be wrong, but whether a given line of reasoning has been completed to a standard that is worth carrying forward. That is a different question, and it requires a different kind of evaluator.

Parcae's explicit acknowledgment that dynamic halting is an open question, combined with the empirical evidence from Kohli et al. and Fu et al. that overthinking is a measurable and consistent failure mode, creates the exact research gap that PTL is designed to fill.

3. What Already Exists and Where the Gap Is

PTL is a new proposed way of thinking about looped reasoning. Every component it draws on has been validated separately. The gap is in their combination.

Adaptive Computation Time (Graves, 2016; Dehghani et al., 2018)

Alex Graves introduced Adaptive Computation Time in 2016, giving recurrent neural networks a learned mechanism to decide how many computation steps to apply before producing an output. Each position accumulates a halting probability, and when that cumulative probability exceeds a threshold, the computation stops. Dehghani et al. extended this to Universal Transformers in 2018. ACT establishes the principle that models should self-terminate based on internal signals rather than fixed schedules. What it does not provide is any graded assessment of the quality of what was computed before halting. The halt gate fires based on learned thresholds, not based on whether the hidden state actually represents a sound and complete resolution of the problem.

Process Reward Models (Lightman et al., 2023; He et al., 2025; She et al., 2025)

Process Reward Models evaluate each intermediate reasoning step rather than only the final answer. OpenAI's foundational paper in 2023 showed that this dense, step-level feedback dramatically improves multi-step reasoning because errors are caught at their source rather than only at the conclusion. The field has expanded rapidly since then. ThinkPRM makes PRMs generative, having

them reason through verification rather than simply classify. Hierarchical Reward Models stack supervision across multiple granularities. R-PRM adds reasoning-driven evaluation for greater reliability. All of these systems operate as training-time supervision signals. None of them operates as an inference-time gate that decides, loop by loop, whether output is ready to be certified and handed forward.

Recursive Reward Modeling and Debate (Christiano, Irving, Amodei 2018)

Paul Christiano and colleagues at OpenAI formalized the principal-teacher hierarchy as a training-time alignment framework in 2018. The structure is layered: a principal supervises teachers, teachers supervise agents. Geoffrey Irving, Paul Christiano, and Dario Amodei developed the related Debate framework, in which two AI agents argue while a judge evaluates truthfulness. Both frameworks established that hierarchical verification is theoretically sound as an approach to scalable oversight. What neither framework proposed was applying this hierarchy at inference time, inside a single forward pass, to verify that the evaluators within the reasoning chain are themselves trustworthy.

Chain of Thought and Scratchpad Memory (Wei et al., 2022)

Chain-of-thought prompting demonstrated in 2022 that language models reason significantly better when intermediate steps are externalized as text. Saunshi et al. (2025) later showed formally that each loop in a recurrent depth transformer is mathematically equivalent to one chain-of-thought step, but operating in continuous latent space rather than over discrete tokens. This is a meaningful result: it means looped reasoning and explicit reasoning are more closely related than they might appear. But existing scratchpad systems carry all intermediate computation forward without quality filtering. There is no mechanism to distinguish a verified conclusion from an uncertain hypothesis. There are no certification stamps. Nothing prevents a degraded loop from polluting the context that subsequent loops build on.

The Gap

Think-at-Hard is the closest existing work to what PTL proposes, and it illustrates the gap precisely. TaH makes a binary per-token decision: iterate more, or stop. PTL proposes a richer system: evaluate the quality of a reasoning unit, distill only what was certified, pass only that forward, and have a separate layer verify that the evaluators are calibrated. These are qualitatively different mechanisms operating at a qualitatively different level of abstraction. No existing unified system combines them.

4. The PTL Architecture

PTL is a new proposed framework that wraps around any Parcae-style looped transformer. It does not replace the base architecture. It adds four layers of reasoning governance to it. Here is how each layer works.

The Loop Agent

The base recurrent block runs each loop with shared transformer weights, following the Anonymous Loops design from Parcae. There are no loop-index embeddings. The model does not know which loop it is on. The LoRA adapter scale is driven by the hidden state norm rather than by loop position, which means the model's behavior at any loop is determined by what the representation looks like, not where it is in the sequence. This is what makes depth extrapolation possible. Each loop agent also receives the previous loop's verified scratchpad alongside the task context, so it does not re-derive what has already been resolved and certified.

The Loop Grader

After each loop, a Process Reward Model evaluates the hidden state output across three dimensions. Coherence asks whether the representation is internally consistent. Completeness asks whether this particular subproblem has been resolved. Confidence asks whether the model's uncertainty estimate is calibrated and trustworthy. These three scores combine into a composite score. If that composite score clears the delegation threshold, the loop's resolved findings are distilled into structured markdown notes and added to the scratchpad with a verification stamp. The next loop does not re-process this content. If the score falls below the halt threshold, the loop runs again with tighter context. If the score lands in an uncertain middle band, the grader flags it for the Principal.

The Structured Scratchpad

The scratchpad grows across loops as a curated record of what has been certified. Each entry records the loop that produced it, the grader scores at the time of distillation, and what was resolved. Only findings that cleared the grader threshold enter the scratchpad. Uncertain or degraded loop outputs are not distilled. The scratchpad is not a dump of all intermediate computation. It is a trust-enforced working memory: everything it contains has been evaluated and stamped. It also provides a complete audit trail that the Principal can read to detect miscalibration in earlier loops.

The Principal Meta-Verifier

The Principal does not evaluate the task output. It evaluates the graders. It observes the confidence scores assigned at each loop and then observes how well subsequent loops performed given the context that was stamped and passed forward. If a grader assigned high confidence to a loop whose output turned out to be misleading, detectable because later loops performed worse than the stamp implied, the Principal flags that grader as miscalibrated. Suspect scratchpad entries can be

down-weighted or rolled back. This prevents grader collapse, a failure mode in which a miscalibrated PRM rubber-stamps poor outputs and poisons the scratchpad with false certainty. The Principal is the system's immune response to that failure.

The Final Synthesis Loop

The last loop receives the full Principal-audited scratchpad. Because every prior loop has been graded, distilled, verified, and checked by the Principal, the final loop operates on the cleanest possible context. It does not re-derive. It synthesizes. This is why the final output can achieve a quality that no individual loop in the sequence could produce alone. The loops beneath it did the work correctly, and the certification hierarchy ensured that correctness was real rather than assumed.

5. Why This Is a New Way of Thinking

PTL is not an incremental improvement on any single prior system. It is a new way of thinking about what a reasoning loop is for. In every existing looped transformer, a loop is a repeated computation step. It runs, updates the hidden state, and passes that state to the next loop. The loop has no accountability. It cannot certify what it resolved. It cannot flag when it is degrading. It cannot hand off selectively. It just runs.

PTL proposes treating each loop as an accountable reasoning agent in a verification chain. The loop does not just compute. It is evaluated. Its output is graded before anything downstream is allowed to depend on it. What it resolved is extracted and stamped. What it failed to resolve is flagged. And the evaluators are themselves watched by a principal that ensures the whole grading process is trustworthy. This is a fundamentally different relationship between computation and verification.

The three novelties that distinguish PTL from everything in the existing literature are the loop delegation protocol, the inference-time principal, and the verified scratchpad.

Loop delegation is distinct from halting. ACT stops a loop from continuing. PTL delegation actively extracts what was resolved, certifies it, and passes it forward as stamped context. The difference matters because a halted loop takes nothing with it. A delegating loop transfers its certified knowledge to the next loop in a structured form that cannot be confused with unverified hypotheses.

The inference-time principal is distinct from all prior uses of the principal-teacher hierarchy. Christiano and Irving built that hierarchy for training. PTL applies it at runtime, during a single inference pass, to audit whether the graders operating inside the pass are calibrated. This is a new application of an established theoretical structure to a new problem.

The verified scratchpad is distinct from existing scratchpad memory. Prior scratchpads carry all intermediate computation forward, certified or not. PTL's scratchpad carries only what was evaluated and stamped. The stamp is not metadata. It is access control. Only certified content can become part of the context the final loop reasons from.

6. Connection to Anthropic's Research

PTL intersects directly with several areas that Anthropic has publicly prioritized, and the timing makes it especially relevant.

On scalable oversight, the Principal layer is a runtime instance of the scalable oversight problem. How do you maintain meaningful supervision over a reasoning process that is more capable than any single evaluator can fully assess? PTL's answer is hierarchical and applies at inference time, not just at training. The graders watch the loops. The Principal watches the graders. This extends the logic of Constitutional AI and weak-to-strong generalization into the inference-time loop regime.

On mechanistic interpretability, the scratchpad makes reasoning structure explicit and inspectable in a way that latent loop computation currently is not. A recent analysis using logit-lens and coda-lens probing found limited evidence for persistent, interpretable latent reasoning across recurrent blocks, with sharp discontinuities rather than stepwise improvement. The scratchpad in PTL would address this directly: every loop's certified output is externalized as structured text, giving interpretability researchers a clean interface to read what each loop resolved and at what confidence.

On test-time compute scaling, PTL provides principled adaptive compute allocation. Easy subproblems resolve in two or three loops and lock their findings. Hard subproblems receive the full loop budget. The final synthesis loop operates on compressed, verified context rather than raw accumulated state. This is a more efficient and more reliable form of test-time scaling than applying a fixed loop count uniformly across all tokens and subproblems.

On the recurrent depth frontier specifically, Parcae was published in April 2026 and is already being discussed as potentially related to Anthropic's internal Mythos architecture. OpenMythos, an open-source reconstruction of that hypothesis, was published days before this paper. The research community is moving fast in exactly the direction PTL addresses. The gap that Parcae left open is the gap PTL proposes to close.

7. A Note on Where This Proposal Comes From

This paper was not written by a researcher with years of published work in machine learning. It was written by someone new to this field who has come to believe that the most valuable thing a person

new to a discipline can contribute is not code but thinking.

The insight at the center of PTL did not come from implementing any of the systems described in this paper. It came from asking a simple question: if a reasoning loop can degrade without knowing it, what would it look like to build a system where every loop is accountable to something beyond itself? That question led to graders. Graders that could themselves be wrong led to the Principal. The Principal needing something to audit led to the scratchpad. The scratchpad needing to carry only what was certified led to delegation. The architecture emerged from reasoning about the problem, not from engineering toward a specification.

The best ideas in any field often come from people who do not yet know what is supposed to be impossible. This proposal is offered in that spirit: as a genuine attempt to think carefully about an open problem, to situate that thinking in the best available research, and to contribute something worth testing even if the person proposing it is still learning how to build it.

8. The Hardest Open Problem: Training the Principal

The most difficult unsolved question in PTL is how to train the Principal. The loop agents can be trained against task correctness with PRM supervision. The loop graders can be trained against whether each loop's output was sound. But the Principal needs to be trained on whether the graders' confidence scores were calibrated correctly, and that requires knowing the ground truth of grader judgment, which is a harder and more abstract supervision signal than any level below it in the hierarchy.

Three candidate approaches exist in the literature. Retrospective calibration would train the Principal on large-scale data recording grader scores and the downstream loop performance that followed each graded stamp. This is tractable but data-intensive and requires extensive rollout infrastructure. Debate-style principal training, following Irving et al., would have two independent graders evaluate the same loop output and train the Principal to distinguish the more trustworthy grader based on downstream outcomes. Weak-to-strong generalization, following Burns et al. (2023), would use a smaller and more trusted model as Principal to supervise a larger and more capable grader.

This paper does not resolve the question. It flags it as the primary barrier to practical PTL deployment and recommends retrospective calibration as the most tractable near-term approach, with debate-style principal training as the more theoretically grounded long-term direction.

9. Conclusion

PTL is a new proposed architecture, offered as a new way of thinking about what looped reasoning can be. It has not appeared in academic literature in its unified form. Its three novel contributions are the loop delegation protocol, the inference-time principal meta-verifier, and the verified scratchpad as a trust-enforced working memory.

The timing is unusually favorable. Parcae just solved the stability problem for looped transformers and explicitly left dynamic halting open. Two independent research groups have proven empirically that overthinking is a real and consistent failure mode in looped architectures. Recurrent depth models are being actively discussed as the next frontier in compute-efficient language model scaling, with direct connections to Anthropic's own research directions. PTL sits at the intersection of all of this: a new proposed model for how the loops inside the next generation of language models can be made not just stable, but accountable, interpretable, and trusted.

The most immediate experimental question is whether per-loop PRM grading produces reliable confidence signals in the depth-extrapolation regime, and whether those signals can drive delegation decisions that genuinely improve final output quality over fixed-loop-count baselines. If they can, PTL provides a concrete and testable path toward recurrent transformers that are not only efficient and stable, but verifiable from the inside.

References

[1] Graves, A. (2016). Adaptive Computation Time for Recurrent Neural Networks. *arXiv:1603.08983.*

[2] Dehghani, M. et al. (2018). Universal Transformers. *arXiv:1807.03819. ICLR 2019.*

[3] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022.*

[4] Lightman, H. et al. (2023). Let's Verify Step by Step. *arXiv:2305.20050. OpenAI.*

[5] Irving, G., Christiano, P., Amodei, D. (2018). AI Safety via Debate. *arXiv:1805.00899.*

[6] Christiano, P. et al. (2018). Scalable agent alignment via reward modeling: a research direction. *arXiv:1811.07871.*

[7] Burns, C. et al. (2023). Weak-to-Strong Generalization. *arXiv:2312.09390. OpenAI.*

[8] Prairie, N. et al. (2026). Parcae: Scaling Laws For Stable Looped Language Models. *arXiv:2604.12946. UCSD / Together AI. April 2026.*

[9] Geiping, J., McLeish, S. et al. (2025). Scaling Up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *NeurIPS 2025.*

[10] Saunshi, N. et al. (2025). Looped Transformers Are Better at Learning Learning Algorithms. *ICLR 2024.*

[11] Kohli, H. et al. (2026). Loop, Think, and Generalize: Implicit Reasoning in Recurrent-Depth Transformers. *arXiv:2604.07822. April 2026.*

[12] Fu, T. et al. (2025). Think-at-Hard: Selective Latent Iterations to Improve Reasoning Language Models. *arXiv:2511.08577. November 2025.*

[13] He, H. et al. (2025). Towards Hierarchical Multi-Step Reward Models for Enhanced Reasoning. *arXiv:2503.13551.*

[14] She, S. et al. (2025). R-PRM: Reasoning-Driven Process Reward Modeling. *arXiv:2503.21295.*

[15] Zhao, J. et al. (2025). GenPRM: Scaling Test-Time Compute of Process Reward Models. *arXiv:2504.00891.*

[16] ThinkPRM (2025). Process Reward Models That Think. *ICLR 2026 Submission.*

[17] Hao, S. et al. (2024). Training Large Language Models to Reason in a Continuous Latent Space. *arXiv:2412.06769. COCONUT.*

[18] OpenMythos (2026). Open-Source PyTorch Reconstruction of Claude Mythos Hypothesis. *MarkTechPost, April 19 2026.*

[19] Jeddi, A., Ciccone, M., Taati, B. (2026). LoopFormer: Elastic-Depth Looped Transformers for Latent Reasoning. *ICLR 2026.*

[20] Lu, Y. et al. (2025). Probing Latent Reasoning in Recurrent-Depth Transformers via Logit and Coda Lens. *arXiv, July 2025.*